OmniFlow: Adaptive On-device Replay System for Mobile GUI Agents

Abstract

abs Modern mobile UIs change between users, devices, and sessions: layouts re-flow, stable selectors are often absent, and content is personalized. LLM-based GUI agents handle this dynamism by re-reasoning at every step—effective on open-ended tasks, but expensive on the recurring routines that dominate real device use. Each step in their reason-act loop may require a fresh model decision, incurring on-device latency, memory, power, and thermal cost, or expanding the privacy boundary when screens are shipped to a cloud planner. Trajectory reuse is the natural serving target, but existing record-and-replay, memory-retrieval, and program-guided systems still fall back to online repair the moment those same dynamic conditions invalidate a stored action.

We present **OmniFlow**, a plug-in cache for mobile GUI agents whose central design is to *check and remap every recalled action against the live screen before it executes*, so historical executions become reusable Functions without requiring identical layouts, selectors, or coordinates at reuse time. **Omni** writes successful run logs into a Function Store whose assets are API-like `Functions`; the store induces sparse UDEG nodes as a privacy-safe state index that keeps raw XML, screenshots, and personal text outside the shared cache boundary. Recall returns compact executable candidates rather than full trajectories. **Flow** drives a recalled Function on the live device through an observe-check-transfer-act loop that checks each action against the current state, remaps the target, executes when safe, and falls back otherwise. Matched Functions therefore skip online reasoning entirely, while uncertain states remain under the host agent through a structured handoff.

On androidworld benchmarks, OmniFlow enables 2.3× reuse of successful trajectories, improving task success rates by 5.4% and reducing latency by 25% on average, with up to 40% improvement on challenging dynamic tasks. User studies show that OmniFlow's reuse leads to smoother interactions and higher satisfaction, especially on tasks with dynamic content. By enabling robust trajectory reuse, OmniFlow significantly enhances the efficiency and user experience of LLM-based GUI agents in the face of mobile UI dynamism.

1 Introduction

Mobile GUI agents powered by vision-language models (VLMs) have demonstrated impressive capabilities in open-ended task execution. However, when deployed in the real world, users rarely utilize these agents solely for open-ended exploration. Instead, users expect them to act as intelligent assistants that accelerate daily, repeated routines–such as checking schedules, filling out common forms, or navigating familiar app workflows–fast and reliably. Currently, mobile GUI agents address these tasks using a standard observe-reason-act loop [1, 17, 20, 22]: observing the current screen, invoking a massive model to decide the next action, executing the operation, and repeating until completion. While this design is highly effective for novel tasks with the support of performant VLMs, applying step-wise VLM reasoning to recurring routines forces every user and every repetition to solve nearly the same interaction sequence from scratch. This repeated model invocation introduces severe computation, memory, energy, and thermal overheads on edge devices. This reality motivates a new serving target for mobile GUI agents: *successful execution histories of daily routines should be efficiently reused to bypass expensive model invocations whenever possible.*

Existing systems have attempted to address this serving target through various approaches, as summarized in Table 1. (1) Per-step acceleration methods [2, 5, 6, 11] reduce the latency of individual model calls but fail to break the expensive observe-reason-act loop. (2) Traditional RPA-style replay and script-based tools [6, 10, 13, 14] eliminate model calls by binding execution to fixed coordinates or fixed UI selectors, yet they are highly brittle to GUI dynamics. (3) Meanwhile, memory-based agents [9, 22, 27] retrieve prior task knowledge but still require online VLM reinterpretation, leaving them too heavy for seamless on-device execution.

Crucially, these prior studies fall short because they cannot accommodate the complex realities of real-world mobile environments, which are driven by two fundamental dynamics. First, **workflow-level dynamics** mean that live tasks frequently diverge from recorded trajectories due to unpredictable interruptions like login screens, pop-up ads, loading delays, or alternative app entry points. A rigid script fails entirely when the exact path breaks. Second, **GUI-state-level dynamics** dictate that even when the overall workflow aligns, the concrete target on the screen may shift due to different device resolutions, font scales, dynamic content, or app updates. Existing approaches mismatch this reality: they are either too expensive by re-planning every step, or too brittle by blindly replaying actions without adapting to shifting GUI states.

To this end, we propose OmniFlow, an adaptive on-device replay system for mobile GUI agents. OmniFlow is designed to seamlessly reuse historical execution fragments while gracefully falling back to VLM reasoning when necessary. The system is built around two core components: (1) the

Omni representation layer, which decomposes past executions into a fine-grained UI transition graph of reusable functions, and (2) the Flow execution layer, which dynamically grounds and verifies recalled actions into the live GUI state before execution.

These two components are specifically designed to solve the aforementioned dynamics through two core technical innovations. First, to tackle workflow dynamics and efficiency, OmniFlow employs a fine-grained transition graph coupled with a fast on-device node matching mechanism. Unlike prior memory systems that rely on computation-heavy visual embeddings, OmniFlow's node encoding relies entirely on feature-engineered page vectors derived from the accessibility-tree structure. This allows for rapid, lightweight node matching executed completely on the mobile device. By matching fine-grained UI states rather than monolithic trajectories, OmniFlow can smoothly alternate between trajectory reuse and VLM reasoning. This maximizes the utilization of stored memories: if a workflow diverges due to a pop-up ad, the system falls back to the VLM to clear it, and instantly resumes replay-based execution once a recognized GUI state is reached.

Second, to address GUI-state dynamics, OmniFlow introduces an anchor-based action transfer design within the Flow execution layer. Instead of executing actions via static coordinates, this mechanism grounds a recorded action in the live GUI by comparing structural, visual, and geometric features between the recorded state and the current screen. It identifies stable neighboring elements as anchors and uses local layout relations and GUI-tree structures to dynamically locate the intended target. If the target match lacks confidence or is blocked by runtime conditions, the system abstains and defers to VLM generation, ensuring robust adaptation to shifting layouts and device variations.

We implement OmniFlow as a plug-in cache and execution layer integrated into mobile GUI agents. We evaluate OmniFlow on the AndroidWorld benchmark, comparing it against xxx baselines. Our experimental results demonstrate that yyy.

This paper makes the following contributions.

- We identify repeated mobile GUI execution as a critical real-world serving workload for VLM-powered GUI agents, and formally characterize the workflow-level and GUI-state-level dynamics that render existing replay-based methods insufficient.
- We design OmniFlow, an adaptive on-device replay system that overcomes mobile GUI dynamics through two cooperating mechanisms: fast on-device node matching on fine-grained execution trajectories to seamlessly alternate between memory reuse and VLM reasoning, and anchor-based action transfer to robustly execute recorded actions on live screens.

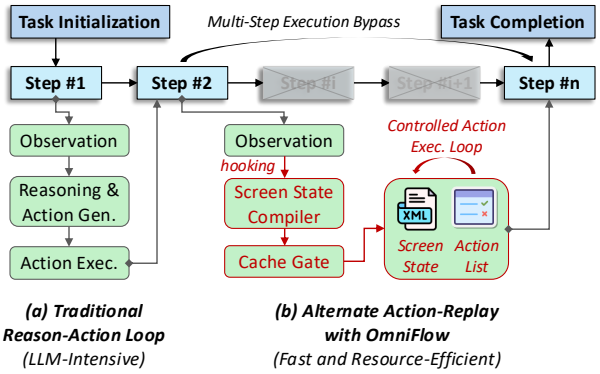


Figure 1. Overview of OmniFlow workflow. The framework dynamically alternates between a traditional LLM-intensive reason-action loop (a) and a fast action-replay mechanism (b) to accelerate task execution. By hooking observations and querying the memory, the system recognizes familiar screen states and adaptively replays a cached list of atomic actions.

- We implement and evaluate OmniFlow on several mobile GUI agents and representative benchmarks. Results show that xxx.

2 Background and Motivation

In this section, we review recent advances in mobile GUI agents powered by emerging LLMs and VLMs, along with the high execution costs associated with the traditional execution pipeline. We then examine efforts to reduce the runtime costs of such agents and discuss their limitations, which motivate the design of OmniFlow.

2.1 High Execution Costs of GUI Agents

Mobile GUI agents typically operate in an observe–reason–act loop [17, 20, 22, 27]. As shown in Figure 1, at each step, the agent perceives the current screen, invokes an LLM or VLM to determine the next GUI action, and executes the action in the environment. Although this workflow enables flexible reasoning over open-ended tasks, it introduces substantial runtime overhead because model inference is repeatedly triggered during long interaction sequences. The cost accumulates with each additional step, leading to increased latency and computation during task execution [1, 5, 26, 28, 29]. Prior work reports that existing mobile agents often require more than 10 seconds per step, 1–2 minutes, and about $1 monetary cost per task on average [4, 5]. More broadly, planning and reflection alone can account for 75%–94% of end-to-end latency in GUI agents [1]. These costs become more pronounced when the step-wise model inference runs directly on mobile devices with strict memory, power, and thermal constraints [4, 5, 25].

However, real-world mobile usage rarely consists of purely open-ended tasks. In practice, users repeatedly perform similar routines, such as checking routine information, retrieving specific content, completing fixed navigation paths, and filling structured form-like workflows. Prior studies characterize smartphone interaction as highly habitual, with repetitive checking behavior and app-launch patterns that are strongly shaped by recurring contexts such as time and location [15, 24]. Public mobile GUI datasets and benchmarks also reflect a similar pattern. For example, the largest scale mobile GUI dataset to date, AITW, contains 715K episodes but only around 30K unique instructions [18]. We further quantify recurrence over the 30,378 subset-level unique-prompt entries reported across AITW's five subsets (Appendix C). Parameter masking alone finds a template peer for 11,052 prompts (36.4%). Under a conservative criterion that accepts either a shared parameter template or semantic similarity of at least 0.98 while rejecting opposite intents, 22,834 prompts (75.2%) have at least one similar counterpart; lowering the semantic threshold to 0.95 raises the estimate to 86.9%. These prompt-level rates estimate the workload available to a Function cache: they characterize candidate reuse demand, while executable Function reuse additionally requires compatible GUI state, action transfer, and terminal-effect verification. Therefore, although per-step reasoning for open-ended tasks is crucial for high task completion rates, many realistic mobile-agent workloads are better viewed as repeated executions of recurring workflow patterns.

[This paragraph is the limitations of existing replay mechanisms; it also introduces massive new concepts such as coordinate replay and selector replay, which have not been introduced ahead.] This reuse problem is constrained by the observation interface itself. Mobile GUI agents usually observe a screenshot, an Android accessibility tree, or both. The accessibility tree reliably exposes element bounds and hierarchy when elements are observable, while semantic attributes such as `resource-id`, `text`, and `content-desc` only appear when developers provide them. As a result, repeated execution cannot assume that every target has a stable selector: coordinate replay breaks under device-size and layout changes, and selector replay fails on missing annotations, custom-rendered widgets, WebViews, obfuscated resource identifiers, and unlabeled container controls. The same instability limits graph-level reuse. A conventional UI transition graph provides a useful state structure, but its nodes are often tied to raw XML or unstable attributes, and its edges record dispatched actions and state transitions rather than verified task effects. The missing object is a callable transition: a UTG edge can record that one action once moved an app from one observed state to another, but it does not by itself package runtime arguments, source-context transfer, or terminal/effect verification as an API-like operation. Such a graph is therefore difficult to reuse as a cross-device and cross-user execution cache under dynamic layouts and user-specific content.

Table 1. Execution acceleration methods for mobile GUI agents. The columns capture whether a method (1) requires prior exploration on GUI states, (2) utilizes a cached memory without invoking LLM/VLM again, and (3) transfers its memory to dynamic and heterogeneous execution environments.

Method	W/ App Exploration	W/ Model Invocation	Cross-Env Transferability
RPA/script replay [6, 10, 13, 14]	✓	✗	✗
Memory-based [9, 22, 27]	✓	✓	✗
Program-guided [19, 21, 23, 29]	✓	✗	✗
Per-step acceleration [2, 5, 11, 16]	✗	✓	✗
OmniFlow	✗	✗	✓

2.2 Reducing Runtime Costs

Prior studies on reducing the runtime costs of GUI agents primarily fall into several categories, which are summarized in Table 1.

Robotic process automation (RPA)-based automation systems treat mobile tasks as fixed workflows by recording and replaying user demonstrations [6, 10, 13, 14]. These systems construct replayable execution traces through human-driven app exploration before task execution. However, the resulting memory is tightly bound to the recorded execution path, GUI layout, and entry state. As a result, even minor changes in the live GUI state, such as screen modifications or layout updates, can cause the recorded execution to fail.

Memory-based GUI agents reduce runtime costs by reusing memory constructed from prior app exploration or first-time task execution. For example, M3A [17] stores prior actions and their reasoning in prompts to guide later execution. AutoDroid [22] and AppAgent [27] construct app-specific memory, such as UI transition graphs, through automated exploration and retrieve this memory during execution. MobileGPT [9] extends this idea with hierarchical app memory that stores reusable procedures and subtasks summarized from prior executions. MobiMem [12] reduces repeated GUI-agent reasoning by reusing prior executions through experience memory, which abstracts task traces into reusable templates, and action memory, which caches low-level action sequences for direct replay. **Program-guided agents** [19, 21, 23, 29] further reduce runtime costs by translating task knowledge or prior demonstrations into executable scripts. These scripts allow repeated GUI operations to be completed through deterministic program execution with fewer model invocations. Together, memory-based and program-guided agents replace parts of VLM-based step-wise action generation with previously validated memory or executable operations. However, these methods still rely on memory constructed from prior app exploration, user

interaction, or first-time execution, which can introduce privacy concerns and limit transfer across users, devices, app versions, and unseen environments.

Another line of studies, **per-step inference acceleration methods**, retain the traditional observe–reason–act loop while reducing the cost of each individual decision. V-Droid [5] replaces auto-regressive action generation with verifier-based ranking over a discretized action space, which substantially reduces latency. ShowUI [11] accelerates VLM inference through UI-guided visual token selection. Other studies accelerate execution by pruning redundant visual or historical context before model inference, such as masking-based GUI element pruning, history compression [2], and position-preserving UI token selection [16].

To summarize, existing approaches still face several limitations. RPA-based systems and script replay provide executable workflows, but they are fragile when the live GUI state diverges from the recorded execution. Memory-based and program-guided agents reduce repeated reasoning, but they often rely on substantial app exploration and provide limited support for workflow-level adaptation. Per-step inference acceleration methods reduce the cost of each individual decision, but they still require repeated VLM-based step-wise action generation in long interaction loops. These limitations motivate us to build OmniFlow for generalizable memory reuse across dynamic app environments.

2.3 Goals and Challenges

Mobile apps dynamically change across users, devices, and sessions and often lack stable APIs for GUI agents to invoke. While traces usually expose a fixed-action subspace: within a ReAct loop, recurring tasks often contain high-frequency repeated Functions, such as opening a search page, entering a query, filling a form, or confirming a save. We call such reusable units `Functions`. This observation suggests storing them as persistent Functions that can be updated over time, so the agent can skip reasoning and execute directly when the same situation recurs.

Our goal is to design a memory system for mobile GUI agents that can reuse past executions in dynamic app environments while allowing flexible switching between memory-guided execution and VLM-based step-wise action generation. This requires memory to support both workflow-level adaptation, where the execution path may change, and state-level adaptation, where a memorized GUI state may appear differently from the live GUI state.

Adaptive Reuse Workflow. The first challenge is representing past executions at a granularity suitable for flexible reuse. Existing memory systems for GUI agents often store a complete long-horizon trajectory or split it into fixed subtasks, then guide the agent to follow the recorded execution path. This design is fragile in real mobile apps, where workflows frequently change due to expired login sessions, pop-ups, loading delays, or permission dialogs. A coarse-grained memory may fail even when most parts of the historical execution remain reusable. This calls for a finer-grained memory representation that can reuse matched Functions while allowing the agent to fall back to VLM-based step-wise action generation when the live workflow diverges.

Table 2. Execution costs of different on-device inference backends.We use Qwen-vl-0.8B-W4A16 on OnePlus 12.

Location	Backend	Input	Tokens	Time
On-device	Llama.cpp	screenshot	~2,500	60 s+
On-device	MNN-LMM	screenshot	~2,500	10 s+
Cloud	Qwen-API	screenshot	~3,200	3 s

Adaptive Action Grounding. The second challenge is applying a memorized operation to a live GUI state safely. Even when the live GUI state corresponds to a memorized GUI state, the target GUI component may shift due to screen size, font scale, list ordering, or dynamic content. This requires the system to be able to utilize historical context to achieve localization enhancement.

3 System Design

3.1 Architecture and Workflow

OmniFlow consists of two layers. (1) Omni representation layer manages history traces and converts them into reusable and callable memory functions. (2) Flow execution layer executes recalled Functions and VLM-generated actions on the live device.

3.2 Omni Representation Layer

Omni exposes two interfaces: *recall* retrieves executable Functions from the Function Store, and *Update* ingests execution logs into the Function Store for offline enhancement.

UDEG. OmniFlow builds a UDEG (Universal-Device-Execution Graph) to support cross-device and cross-user Function reuse. UDEG nodes are clusters of GUI states, and edges are executable Functions.

Some works use UTG (UI transition graph) to represent the app state space, but mobile GUI states are often highly dynamic and user-specific, which makes UTG need to be updated frequently. Instead of storing raw screenshots or raw XML, UDEG nodes store compact feature vectors that capture the visual, structural, geometric, and interaction cues of the GUI state. OmniFlow uses xml-feature compactor to acomplish page embedding because vision-embedding-model is expensive and privacy-sensitive, while xml features are cheap and structured. We prove in §5 that this representation is sufficient for high-accuracy page matching and Function recall.

For each element, it builds a 64-dimensional feature vector from two sources: screenshot-derived visual cues, such

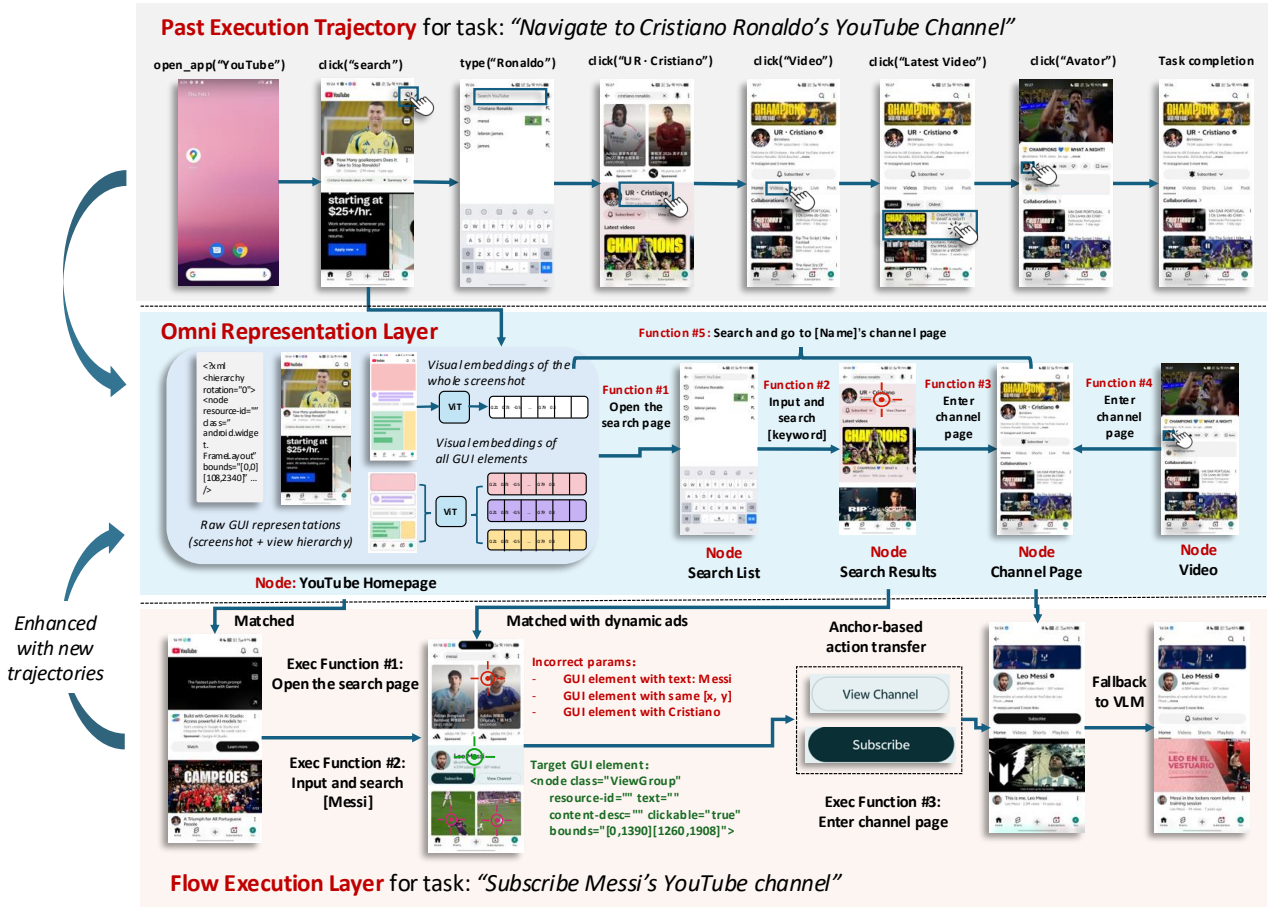


Figure 2. Demonstration of how a task execution trajectory is transformed into memory and how that memory is adaptively utilized in online execution environments.

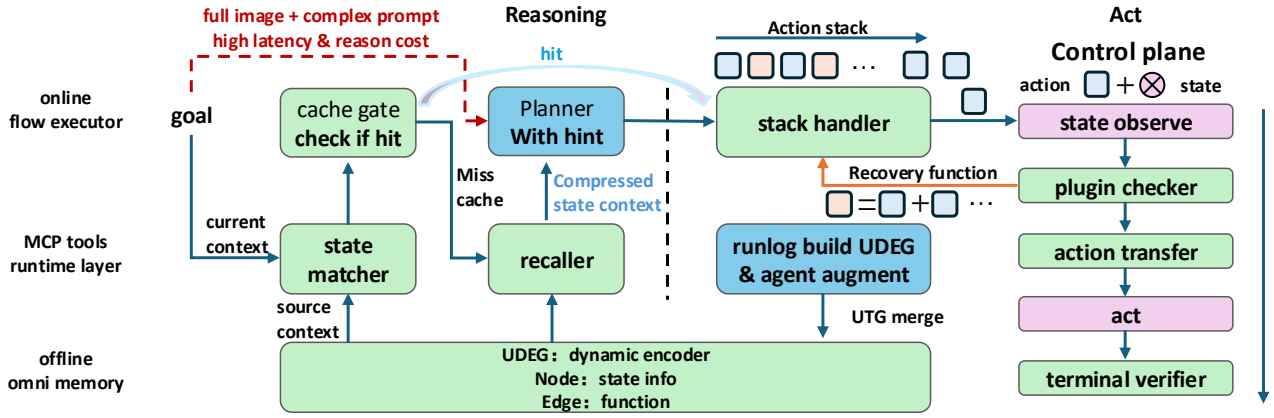


Figure 3. System design overview. OmniFlow builds a Function Store whose assets are API-like Functions synthesized from successful GUI executions. The store induces a sparse UDEG: privacy-safe GUI-state nodes index where Functions can be invoked, and Function edges record executable task effects. Omni recalls a compatible Function through this state index; Flow executes it through a single observe-check-transfer-act stack and writes accepted run logs back into the Function Store.

as icons, labels, and clickable regions, and view-hierarchy-derived cues, such as widget class, interaction flags, bounds, and tree position.

The element vectors are pooled into a compact page vector for approximate lookup. For each UI element e_i, OmniFlow builds an element vector

$$\mathbf{v}_i = \phi(e_i),$$

where $\phi(\cdot)$ combines visible cues and accessibility-structure cues: content hash, element type, interaction flags, subtree shape, hierarchy, resource-id hints, and normalized geometry. OmniFlow then groups elements into UI subspaces, each defined by a predicate over the element and its page context:

$$\mathcal{S}_k(X) = \{e_i \in X \mid c_k(e_i, X) = 1\}.$$

For each subspace, OmniFlow computes a weighted vector:

$$\mathbf{p}_k(X) = \frac{\sum_{e_i \in \mathcal{S}_k(X)} \alpha_{ik} \mathbf{v}_i}{\sum_{e_i \in \mathcal{S}_k(X)} \alpha_{ik}}.$$

The page vector is the weighted concatenation of these subspace vectors:

$$\mathbf{p}(X) = \frac{[w_1\mathbf{p}_1; \ldots; w_K\mathbf{p}_K]}{W_P}, \qquad W_P = \sum_{k=1}^{K} w_k.$$

This method can quickly locate a matching page in the UDEG, which enables two key capabilities: (1) recalling Functions that can be invoked in the current GUI state, and (2) efficiently retrieving similar historical actions from the matched page to enhance the current operation. In §5, we demonstrate that this approach achieves high top-1 accuracy in page matching.

Recall. OmniFlow uses the current GUI state and task context to recall candidate Functions from the Function Store. When an observe cycle completes, Omni queries the UDEG index to retrieve a set of potentially invokable Functions. These Functions are then passed to a lightweight cache gate model that assesses their executability and fills in any required parameters. This approach leverages general knowledge for semantic similarity assessment and parameter completion, allowing the small model to perform effectively. Typically, a single invocation requires only a few milliseconds, enabling rapid decision-making in real-time execution scenarios.

Update. Updater ingests execution logs into the Function Store to enhance the executability and generalization of Functions. Upon receiving a raw execution trace, it is first processed by an enhancement module that performs offline augmentation. Through backend agent calls, key actions are identified, semantic information of the actions is extracted, and structured documentation is generated for subsequent model deployment. This process ensures that the Function Store remains up-to-date with the latest execution patterns and can effectively support future task executions.

3.3 Flow Execution Layer

Flow is the execution layer that grounds recalled Functions and VLM-generated actions on the live device. It provides a action stack for pending actions, and execute each action through a observe-checker-transfer-act loop.

When an action arrives, we enhance its execution performance using two components. First, we utilize checkers to distill the agent's observations of past execution history into executable rules, which are then validated in the new environment. Second, we employ transfer to relocate existing actions to the current context.

Plugin checkers. During execution, the action sequence is not fixed and may be interleaved with interrupt actions, such as closing pop-ups or dismissing keyboards. Previous works often require fallback or manual intervention for such cases, but local errors can be managed through a unified set of rules. To enable local recovery for a Function, we implement checkers that serve as a middleware, converting natural language and prompts into fixed actions. This allows OmniFlow to continuously create new checkers for Functions and nodes during offline enhancement, capturing error signals and reusing them during actual execution.

On execution, each checker defines a configurable trigger over the current GUI state, the pending action, or the current execution context. When the trigger condition is met, the checker can either inject a corrective action into the pending action stack or abort the current Function execution. This mechanism allows for dynamic adaptation to unexpected GUI changes, ensuring that the execution remains robust and reliable even in the presence of environmental variability.

Action transfer. Traditional coordinate replay and selector replay rely on a single unstable consumption: the recorded point or the target element's own attributes. This is fragile on mobile GUIs, where elements often lack stable `resource-id`, visible text, or accessibility names, and where layouts shift across executions.

OmniFlow instead treats transfer as relation-aware matching between the source and current context. Transfer maps a recorded source action in a recalled `Function` to an executable action on the live GUI state:

$$\texttt{transfer} : (S, e, T) \mapsto e',$$

where S is the recorded source page, e is the source action target, T is the current page, and e' is the target element to execute on the live device. The design follows a common property of mobile GUI tasks.

It scores source and current elements with element vectors, selects high-confidence one-to-one matches as anchors, and lets each anchor vote for a compatible current target by projecting the recorded action through the action–anchor relation. Each vote combines action similarity, anchor similarity, and action–anchor distance consistency from bounds and parent links. This makes transfer depend on stable local and global UI structure, such as cards, search bars, tab bars, section headers, and navigation controls, while avoiding an early commitment to global anchors that do not explain the current action.

4 Implementation

OmniFlow is implemented in both Kotlin and Python so the cache layer can be deployed at different integration points

Algorithm 1 Anchor-based action transfer for one recorded action

Require: source page S; current page T; recorded source action element e
Ensure: executable target action or ABSTAIN

$M \leftarrow$ pairwise element-similarity matrix between S and T
$A \leftarrow$ high-similarity one-to-one pairs (a, a') from M
$V \leftarrow$ empty vote table over elements in T
for all $(a, a') \in A$ **do**
  $\hat{x} \leftarrow$ project e through the path $e \rightarrow a \rightarrow a'$
  $C \leftarrow$ compatible current elements near $\hat{x}$
  for all $e' \in C$ **do**
    $r \leftarrow$ relation consistency between (e, a) and (e', a')
    $v \leftarrow \text{SIM}(e, e') \cdot \text{SIM}(a, a') \cdot r$
    $V[e'] \leftarrow V[e'] + v$
  end for
end for
$c^* \leftarrow$ highest-vote element in V
if c^* has enough vote support and margin **then**
  return recorded action rebound to target element c^*
else
  return ABSTAIN
end if

without changing the host agent. The Kotlin implementation runs close to Android: it binds to the accessibility service and input APIs, collects observations, dispatches physical actions, and can host the local UDEG index on device. The Python implementation targets host-side agents, evaluation harnesses, and shared stores: it exposes the same Function recall, Function execution, and run-log ingestion tools while hosting the shared UDEG index and Function store. Both implementations use the same `run_log` schema, `Function` format, and MCP tool definitions, so a run log collected from a human, a VLM agent, or a benchmark harness enters the same write-back path.

A central integration goal is zero modification to the host agent. OmniFlow attaches at the physical GUI I/O boundary—`observe`, which reads the current screen state, and `act`, which dispatches one device action. Everything above this boundary remains the host system's planner and task loop. The three MCP tools wrap this boundary into a cache layer: *Function recall* intercepts before planning, *Function execution* drives the device after a Function is selected, and *run-log ingestion* writes accepted executions back after completion. The same tool boundary is the measurement surface used in §5.

We collect run logs from both human and VLM agents. Human run logs are collected by a custom Android app that records the accessibility tree, screenshots, and input events during a user session. Different actions are recorded with different input events, such as touch events for taps and swipes, text input events for typing, and key events for hardware buttons. The app also records the timestamps of each action to capture the temporal aspect of the execution trace.

Table 3. AndroidWorld E2E report

System	SR	Total time	Calls	Tokens
M3A (Qwen-VL)	70.0%	146.4 min	415	2,120,446
AutoDroid	36.7%	64.9 min	81	62,029
MobileGPT †	53.3%†	85.2 min	576	476,257
OmniFlow	**100.0%**	**6.1 min**	14	5,275

5 Evaluation

5.1 E2E replay experiment

Tasks Setting. The comparative study measures whether a human-collected source execution can be reused on similar tasks with lower latency and online model cost.

We use AndroidWorld as a validator. This set contains 30 accepted replay task instances covering system-level apps and settings, such as SMS/Phone, Contacts, Camera, Recorder, Clock, Bluetooth, Wi-Fi, and brightness controls. Each source execution is successfully recorded, and each reuse execution is scored by the AndroidWorld validator with a different replay seed and different device from the source run.

We use qwen3-vl-plus as agent backend for all systems, including the baselines, to ensure a fair comparison of the reuse strategies rather than differences in underlying model capabilities.

Baselines. For each method, we record and provide a raw runlog as human-teacher. Methods only know tasks' descriptions. We choose these methods as our baseline: (1) A simple RPA baseline. We

(2) AndroidWorld baseline. We use a M3A Androidworld original implementation with hint.

(3) AutoDroid. It builds a reusable transition structure from the source run and attempts to reuse actions through UI selectors and observed app transitions. This baseline tests whether selector-level reuse is sufficient when GUI states shift. (4) MobileGPT. It convert source runlog as subtasks and reuse them.

Metrics. The primary metrics are task-level success rate (SR), average fallback count per run (the number of steps that could not be served from cache and fell back to the host agent), total token consumption, and end-to-end latency.

We calculate the average tool call and token overhead of successful examples to balance the large amount of error caused by incorrect exploration. We found that we not only achieved a TOP-1 success rate. It also achieves minimal token overhead. In most jobs, Our OmniFlow's cache gate successfully recalls the correct reusable state and function for all tasks. while parameters slotting is necessary and not

Table 4. Device pool for on-device replay evaluation.

Device	OS Family	Display	Resolution	Form Factor
OnePlus 12	OxygenOS	6.82"	3168×1440	Phone
Pixel 9 Pro	Android	6.3"	2856×1280	Phone
vivo X300 Pro	OriginOS	6.78"	2800×1260	Phone
HONOR Magic V5	MagicOS	7.95"	2352×2172	Foldable

important component for GUI tasks. On execution we do not use any other model calls.

On the other hand, other methods requires frequent calls to embedding models, fallback models, etc. Without checker and source context retrieval, they often failed on each easy tasks.

5.2 On-Device Replay Workload

AndroidWorld primarily covers system built-in app tasks. To evaluate the deployment setting targeted by OmniFlow, we also prepare an app-level reuse workload on real devices. This workload contains repeated workflows in common third-party black-box apps, such as searching, opening a merchant page, selecting an item, dismissing popups, filling a form, and navigating dynamic list or card interfaces. These cases stress WebView pages, account-specific content, personalized lists, popups, and layout shifts that are less central in system-app benchmarks.

The on-device experiment asks whether OmniFlow can preserve the replay service model when the host agent and auxiliary models run under realistic device constraints. We compare OmniFlow with on-device variants of VLM-driven agents and memory-based replay systems. The key measurement is not only task success, but also whether each method repeatedly invokes screenshot/XML understanding models during execution.

For each task, we record one source execution on one device and replay the extracted Function on held-out devices. We measure task success, end-to-end latency, model calls, token consumption, active device time, and resource counters such as CPU time, memory, and battery delta when available. This protocol tests the central systems claim: OmniFlow moves most replay work into structured Function recall, PageVector matching, and control-aware execution, so repeated screenshot/XML reasoning is avoided on the hot path.

5.3 Ablation Study

UDEG accuracy. We evaluate the representation layer independently from final task execution. The goal of this experiment is to test whether UDEG can group cross-device observations of the same action-compatible GUI state and retrieve the correct cluster for a held-out page.

TODO: on-device component cost case study

Plot per-system or per-stage cost for one representative replay run.

Suggested stages: Function recall, PageVector match, action transfer, device I/O, verifier, and fallback model calls.

Figure 4. Placeholder for the on-device replay cost breakdown. Fill this figure with measured latency, active device time, and energy/resource counters after the physical-device protocol is frozen.

We compare two representation methods: (1) a screen-only VL embedding baseline that encodes the raw screenshot using a vision-language model, and (2) our designed UDEG/PageVector method that combines visual and structural information. The results show that our method significantly outperforms the baseline, achieving a much higher cross-device representation recall. This is because the baseline only captures the visual features of the screen, while our method captures both visual and structural information, allowing it to more accurately identify similar page states across different devices and achieve higher cross-device reuse rates. In 10k node UDEG, one-time localization recall use only

action-transfer accuracy. Dataset construction. We evaluate action-transfer on a public cross-platform widget-mapping dataset, converted into a unified source-conditioned grounding format. Each example contains a recorded source screen, the source UI element operated by the trace, a target replay screen, a set of target UI candidates, and one gold target element. We do not manually create synthetic labels. Instead, we use the published widget correspondences as gold source-target grounding pairs. For each target screen, candidates are extracted from the UI hierarchy/accessibility XML, including text, content description, resource id, class, action flags, and normalized bounding boxes. The source side is represented by the operated element and its local context. If multiple extracted candidates correspond to the same public gold widget, for example an XML node and a public mapped widget with the same bounds, they are treated as equivalent correct targets.

This produces 6,730 source-target grounding queries with 265,590 target candidates, averaging about 39 candidates per query. The dataset is split into train, validation, and test subsets. Training queries are used only to fit the structured action-transfer ranker and select fusion weights on validation data; all headline numbers are reported on the held-out

Table 5. Placeholder for on-device end-to-end replay and resource accounting. All values should be filled from the same task set, device pool, account state, and measurement protocol.

System	Mode	Source context	SR	Latency	Model calls	Tokens	Active time	Battery / energy
OmniFlow	Function replay	run_log → function asset	TBD	TBD	TBD	TBD	TBD	TBD
UI-TARS-2	End-to-end GUI agent	none / prompt / demo	TBD	TBD	TBD	TBD	TBD	TBD
Mobile-Agent-v3.5 (GUI-Owl)	End-to-end GUI agent	none / prompt / demo	TBD	TBD	TBD	TBD	TBD	TBD
Mobile-Agent-v2	End-to-end mobile agent	none / prompt / demo	TBD	TBD	TBD	TBD	TBD	TBD
OS-Atlas	GUI action model	none / prompt / demo	TBD	TBD	TBD	TBD	TBD	TBD
AutoGLM-Phone	Phone GUI agent	none / prompt / demo	TBD	TBD	TBD	TBD	TBD	TBD
MobileGPT	Memory-based replay	source run log / memory	TBD	TBD	TBD	TBD	TBD	TBD
M3A (Qwen-VL)	Step-wise VLM agent	task prompt / hint	TBD	TBD	TBD	TBD	TBD	TBD

Table 6. Cross-device representation recall and encoding cost.

Method	Hit@1	Hit@3	Avg encode time	Avg tokens
Qwen-VL-embedding	69.44%	87.50%	188.69 ms	1,009
PageVector	98.63%	100.0%	4.13 ms	0

test split. Screenshots are retained for inspection and possible visual baselines, but our deployed action-transfer model only uses structured UI evidence from XML and metadata, matching the on-device replay setting.

Evaluation. The task is candidate-level grounding, not action prediction. Given a source element and a target candidate set, the model outputs a ranked list of target candidates. The primary metric is Top-1 grounding accuracy:

$$\text{Top1} = \frac{1}{N}\sum_{q=1}^{N} \mathbf{1}\left[\arg\max_i s_\theta(q, c_i) \in \mathcal{G}_q\right],$$

where $s_\theta(q, c_i)$ is the action-transfer score for candidate c_i, and $\mathcal{G}_q$ is the set of gold-equivalent target candidates. We also report average and p95 latency, because the matcher is used in the replay hot path. A method is considered better only if it improves Top-1 accuracy without exceeding the latency budget required for interactive replay. In addition to aggregate accuracy, we analyze failure cases by candidate ambiguity, duplicated labels, weak source text, layout drift, and missing local context. This lets us distinguish true grounding errors from dataset ambiguity and shows whether the method is robust under realistic UI changes.

6 Discussion

Execution flywheel: every run refines the shared cache. A key property of OmniFlow is that the shared Function cache improves with natural usage—no dedicated exploration phase is required. Each accepted run is written back through the same write-back path: Omni projects the new observation into the existing UDEG node family, adds the run as additional evidence, and refines the Function's precondition and action evidence offline. The consequence is that the same UDEG node accumulates observations from diverse devices, users, app versions, and screen configurations over time. A Function that was originally recorded on one device and screen size gradually acquires anchor evidence from foldable displays, different OEM skins, and varying font scales. This makes subsequent cache hits more robust—not because the system was trained on more data, but because the PageVector and anchor evidence naturally span a wider layout manifold. The cost structure follows: the per-Function online model cost is paid once, on the first execution that discovers the Function. Every subsequent cache hit amortizes that cost across all future users. In workloads where the same recurring Function is triggered by many users, the marginal online model cost per invocation approaches zero as the cache matures. This flywheel is absent in existing memory-based systems that build per-user, per-app memory from dedicated exploration: each new user resets the cold-start cost, and the memory does not cross-fertilize across users or devices.

Our efficient page matching and anchor relocation algorithms make repeated write-back practical at runtime. As a result, every successful execution can be incorporated into the shared cache with little additional overhead, allowing execution knowledge to accumulate continuously through normal usage.

7 Related Work

Step-wise and verifier GUI agents. VLM-based mobile GUI agents usually follow an observe–reason–act loop: each screen update is converted into a new model decision, and the system issues one concrete GUI action before observing again. V-Droid [5] improves this loop by shifting the LLM role from action generation to action verification over a discrete action set, reporting 4.3 s per step and 59.5% success on AndroidWorld. VeriSafe [8] similarly focuses on action safety by checking LLM-generated actions against logic constraints derived from the user's goal. These systems improve the cost or correctness of individual steps, but repeated routines still pass through online decision or verification for each interaction. OmniFlow is complementary: a step-wise or verifier-based agent remains the fallback, while matched repeated Functions are served as executable Function cache hits.

Memory-based and programmatic GUI agents. AutoDroid [22], MobileGPT [9], and AppAgent [27] reduce repeated GUI-agent work by building app- or task-specific memory from exploration and prior executions. AndroidWorld's M3A baseline [17] also exposes the benefit of carrying previous actions and reasoning into future attempts. These systems reduce per-goal LLM cost when a warm-start match is found, but their memory is primarily used to guide online reasoning or replay app-specific subtasks. Reuse is limited by per-app cold-start exploration and UI-attribute-based action grounding: XML selectors break on WebViews, custom-rendered feeds, and cross-device layout variation, which forces repair, new exploration, or online LLM calls. ActionEngine [29] and AutoDroid-V2 [23], as well as AgentProg [19], move toward programmatic execution, building app-level state knowledge or generated scripts to reduce online planning. Their reuse unit is a broad app model or complete task program that requires substantial upfront exploration and assumes a stable live app structure. OmniFlow takes a narrower boundary: it treats a verified source run as the asset, caches a Function-level action list, and focuses on transferring each recorded source action to the target GUI instance.

GUI grounding. Visual grounding models for GUI, such as SeeClick [3], UGround [7], and ShowUI [11], train VLMs to map a natural-language element description to a screen coordinate or bounding box. These models optimize for the *zero-prior* setting: given only a task instruction and the current screenshot, they locate the target element without any recorded demonstration. This makes them general-purpose and immediately applicable to novel apps and screens. OmniFlow addresses the complementary setting: a human demonstration already exists, and the goal is to rebind its recorded actions to the current screen without a model call. GUI grounding models could in principle serve as the transfer oracle in OmniFlow's execution loop, but their per-step inference cost reintroduces the model overhead that OmniFlow is designed to eliminate. Feature-engineered element vectors let Flow transfer recorded targets through structural similarity alone, reserving model calls for genuinely ambiguous or novel states.

8 Conclusion

We presented OmniFlow, an adaptive replay system for mobile GUI agents that turns past successful executions into reusable `Functions`. The core design separates reusable execution knowledge from live execution control. Flow applies those Functions on the live device by observing the current state, checking pending actions, transferring recorded targets to current screens, and falling back when the evidence is insufficient.

This representation/execution split changes the cost model of repeated GUI serving. A successful run can write back a reusable `Function`; later compatible users, devices, or task variants can execute the matched Function without re-planning it. Cache hits skip online reasoning for the matched Function, while misses and unsafe transfers return compact page and execution context to the host agent. The online cost is therefore amortized over newly discovered reusable Functions rather than over every user, goal, and repetition.

OmniFlow currently focuses on Android GUI execution and still depends on sufficient state similarity for safe reuse. Large UI redesigns, highly dynamic content regions, and true cold-start goals require fallback to the host agent and later write-back. Its adaptation also has a clear scope: Flow can transfer source actions across current GUI states when structure, geometry, and effect evidence support a confident target, while open semantic decisions can remain VLM, script, or host-agent boundaries. This makes OmniFlow a practical adaptive replay layer: executable, privacy-safe `Functions` are shared, transferred, checked, and refined from future run logs.

A Page-Vector Encoding: Formal Specification

This appendix formalizes the page-vector construction used by the UDEG representation. The purpose of the encoding is to turn a mobile observation into a compact state key that supports fast lookup and action relocation, without storing raw screenshots, raw XML, or source-screen text in the shared payload.

Element vector. Let e_i denote the i-th element in an accessibility tree with n elements. OmniFlow maps e_i to a 64-dimensional vector $\phi(e_i) \in \mathbb{R}^{64}$, formed by concatenating two feature blocks:

$$h(e_i) = [c_i; t_i; a_i; m_i; s_i] \in \mathbb{R}^{16+4+4+4+4},$$
$$g(e_i) = [k_i; b_i; u_i; \ell_i; d_i] \in \mathbb{R}^{8+8+12+2+2}.$$

Here c_i is a text or visual-content hash, t_i is content type, a_i is affordance, m_i is visual prominence, s_i is visual state, k_i is widget class, b_i is boolean attributes, u_i is bounded subtree structure, ℓ_i is hierarchy position, and d_i is the resource-id hint. The final embedding is:

$$\phi(e_i) = [\lambda_h\, \nu(h(e_i))\,;\, \lambda_p\, \nu(g(e_i))], \quad \nu(\mathbf{x}) = \begin{cases} \mathbf{x}/\|\mathbf{x}\|_2, & \|\mathbf{x}\|_2 > 0, \\ \mathbf{0}, & \text{otherwise.} \end{cases}$$

The current implementation uses $\lambda_h = 0.55$ and $\lambda_p = 0.45$.

Human-view block. The human-view block $h(e_i) \in \mathbb{R}^{32}$ encodes rendered-UI cues: a 16-dimensional content hash, 4 content-type dimensions, 4 affordance dimensions, 4 visual-prominence dimensions, and 4 visual-state dimensions.

Programmer-view block. The programmer-view block $g(e_i) \in \mathbb{R}^{32}$ encodes accessibility-XML cues: an 8-dimensional widget-class hash, 8 interaction and state flags, a 12-dimensional bounded-subtree hash, 2 hierarchy dimensions, and 2 resource-id hint dimensions. Raw `text` and `content-desc` strings are

Algorithm 2 XML-to-page vector encoding

Require: XML observation X and optional screenshot I
Ensure: normalized page embedding $\mathbf{p}$

$E \leftarrow \textsc{ParseElements}(X)$
for all $e_i \in E$ **do**
  $\mathbf{h}_i \leftarrow \textsc{HumanView}(e_i, I)$
  $\mathbf{g}_i \leftarrow \textsc{ProgrammerView}(e_i, X)$
  $\phi(e_i) \leftarrow [\lambda_h \nu(\mathbf{h}_i); \lambda_p \nu(\mathbf{g}_i)]$
  $\omega_i \leftarrow \textsc{ElementWeight}(e_i, X)$
end for
for $r = 1$ to 8 **do**
  $E_r \leftarrow \{e_i \in E \mid \textsc{Slice}(e_i, X) = r\}$
  if $E_r = \emptyset$ **then**
    $\mathbf{p}_r \leftarrow \mathbf{0}$
  else
    $\bar{\mathbf{p}}_r \leftarrow \frac{\sum_{e_i \in E_r} \omega_i \phi(e_i)}{\sum_{e_i \in E_r} \omega_i}$
    $\mathbf{p}_r \leftarrow \textsc{NormalizeSlice}(\bar{\mathbf{p}}_r)$
  end if
end for
$\mathbf{p} \leftarrow [w_1\mathbf{p}_1; \ldots; w_8\mathbf{p}_8]$
$\mathbf{p} \leftarrow \textsc{NormalizeForLookup}(\mathbf{p})$
return $\mathbf{p}$

not serialized in the minimal shared payload. Local XML-built pages may keep hashed text and selector diagnostics for action mapping, but these fields are opt-in diagnostics rather than the default shared page-vector payload.

Slice pooling. OmniFlow pools element vectors into eight page slices. The current slice order is: R_1 = global text, R_2 = top-bar text, R_3 = footer/navigation text, R_4 = action text, R_5 = state text, R_6 = global UI structure, R_7 = body structure, and R_8 = front-surface layout. The top-bar and footer cuts are quantile-based rather than fixed 10% screen bands, so the same encoder can adapt to pages with different root heights and system UI insets.

For each slice R_r, the page-level vector is pooled with state-aware element weights:

$$\bar{\mathbf{p}}_r = \frac{\sum_{e_i \in E_r} \omega_i \phi(e_i)}{\sum_{e_i \in E_r} \omega_i},$$

where E_r is the set of elements assigned to the slice and ω_i is a positive element weight. If E_r is empty, $\bar{\mathbf{p}}_r = \mathbf{0}$. Each nonzero slice is normalized to $\hat{\mathbf{p}}_r$.

The page-level recall vector is the slice-weighted concatenation:

$$\mathbf{p} = [w_1\hat{\mathbf{p}}_1; \ldots; w_8\hat{\mathbf{p}}_8] \in \mathbb{R}^{512},$$

with fixed slice weights w_r. The vector store normalizes this final vector to unit length for cosine lookup.

Page similarity. The similarity between two pages is computed over normalized page-level vectors:

$$\mathrm{sim}(\mathbf{p}^{(1)}, \mathbf{p}^{(2)}) = \mathbf{p}^{(1)} \cdot \mathbf{p}^{(2)}.$$

This fixed 512-dimensional vector is used for fast candidate recall. The serialized page-vector payload used by action transfer is larger because it also contains element vector records, root bounds, and element bounds. The evaluation therefore reports the actual recalled payload size emitted by the runner instead of assuming a constant page-vector size.

Privacy invariant. The encoding pipeline satisfies the following operational invariant: *the minimal shared page-vector payload contains no raw text or raw content-description string from the source accessibility tree.* Element records retain bounds because action relocation needs geometry. Local diagnostic payloads may additionally include resource IDs, class names, flags, and text/content-description hashes when action mapping needs exact signature evidence. The default shared serialization omits those identity-bearing fields. This is an operational privacy boundary, not a formal differential-privacy claim.

B On-Device Observation Description Cost

Table 7 records the appendix-only diagnostic snapshot used to contextualize on-device memory lookup cost. The snapshot measures the time and token scale for converting one mobile observation into a complete textual description. It is not an end-to-end task result, a success rate, or an energy measurement. Its role is to show why repeated screenshot/XML description is a bottleneck for an on-device replay system, and why OmniFlow uses PageVector lookup as a 0-token state key.

Table 7. Appendix-only single-observation description cost snapshot.

Location	Backend	Input	Tokens	Time
On-device	Llama.cpp + 0.8B-W4 + CPU	screenshot	~2,500	60s+
On-device	MNN-LMM + 0.8B-W4 + CPU	screenshot	~2,500	10s+
On-device	Omniinfer + 0.8B-W4 + NPU	XML	~1k–10k+	3s
Cloud	Qwen-VL-Plus	screenshot	~3,200	3s
OmniFlow	PageVector	XML features	0	4.13 ms/item

C AITW Prompt Similarity Characterization

Dataset and denominator. We use the prompt vocabulary of Android in the Wild (AITW) [18]. AITW reports unique-prompt counts separately for General, GoogleApps, Install, Single, and WebShopping. We preserve this official five-subset accounting, so the denominator is the sum of the five counts, 30,378. Merging identical raw strings across subsets yields 30,165 globally distinct strings; the difference comes from 213 additional cross-subset occurrences. We extracted the strings through a processed public index[1], which covers 5,687,349 interaction steps and 714,327 episodes. Its prompt vocabulary matches the official count in every subset. We form candidate peers over the global prompt pool and

[1] https://huggingface.co/datasets/KMK040412/aitw-processed-labeled-full

Table 8. AITW prompt-level reuse-candidate estimate. The main paper uses the strict $\tau = 0.98$ result.

Criterion	Similar / Total	Rate
Parameter template only	11,052 / 30,378	36.4%
Template or cosine ≥ 0.98	22,834 / 30,378	75.2%
Template or cosine ≥ 0.95	26,386 / 30,378	86.9%

Table 9. Strict AITW result by official subset ($\tau = 0.98$).

Subset	Similar / Total	Rate
General	69 / 545	12.7%
GoogleApps	81 / 306	26.5%
Install	549 / 688	79.8%
Single	9,151 / 15,366	59.6%
WebShopping	12,984 / 13,473	96.4%

project the resulting indicator back to the official subset-level entries.

Parameter templates. We canonicalize case and punctuation, then replace URLs, email addresses, quoted values, and numeric values with typed placeholders. This operation groups instructions that express the same operation with different task arguments. A prompt is counted by the template-only statistic when another prompt maps to the same canonical template.

Semantic matching. We encode each globally distinct prompt with `BAAI/bge-small-en-v1.5` and compute cosine similarity between normalized embeddings. For threshold τ, prompt p is marked as having a reuse candidate when at least one other prompt q satisfies

$$M_\tau(p) = \mathbf{1}\left[\begin{array}{ll} \exists q \neq p : & T(p) = T(q), \quad \text{or} \\ & \cos(z_p, z_q) \geq \tau \wedge \neg C(p, q) \end{array}\right],$$

where $T(\cdot)$ is the parameter template, z is the prompt embedding, and C is an explicit contradiction filter. The filter rejects pairs that reverse the requested operation, including on versus off, enable versus disable, install versus uninstall, add versus remove or delete, create versus delete, open versus close, show versus hide, login versus logout, subscribe versus unsubscribe, allow versus block, accept versus reject, and increase versus decrease. We report

$$R_\tau = \frac{1}{30{,}378} \sum_p M_\tau(p).$$

The main analysis uses $\tau = 0.98$; $\tau = 0.95$ is reported only as a sensitivity point.

Scope. The unit above is a prompt, so R_τ is a Function-reuse candidate upper bound rather than an executable-reuse measurement. The analysis does not establish that paired episodes share a compatible start state, action sequence, argument binding, or terminal effect.

Trace test. A separate AITW trace-correspondence protocol can sample distinct source and target episodes from the same app and parameter template, add same-app near-miss and opposite-intent hard negatives, and split by held-out templates. Function Recall@k, segment-alignment accuracy, and abstain rate then measure offline generalization. End-to-end replay success still requires a live environment with control-aware execution and terminal/effect verification.

References

[1] Reyna Abhyankar, Qi Qi, and Yiying Zhang. 2025. Osworld-human: Benchmarking the efficiency of computer-use agents. *arXiv preprint arXiv:2506.16042* (2025).

[2] Gongwei Chen, Xurui Zhou, Rui Shao, Yibo Lyu, Kaiwen Zhou, Shuai Wang, Wentao Li, Yinchuan Li, Zhongang Qi, and Liqiang Nie. 2025. Less is more: Empowering gui agent with context-aware simplification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5901–5911.

[3] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. *arXiv preprint arXiv:2401.10935* (2024). https://arxiv.org/abs/2401.10935

[4] Filippos Christianos, Georgios Papoudakis, Thomas Coste, Jianye HAO, Jun Wang, and Kun Shao. 2025. Lightweight Neural App Control. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=BL4WBIfyrz

[5] Gaole Dai, Shiqi Jiang, Ting Cao, Yuanchun Li, Yuqing Yang, Rui Tan, Mo Li, and Lili Qiu. 2025. Advancing mobile gui agents: A verifier-driven approach to practical deployment. *arXiv preprint arXiv:2503.15937* (2025).

[6] Lorenzo Gomez, Iulian Neamtiu, Tanzirul Azim, and Todd Millstein. 2013. Reran: Timing-and touch-sensitive record and replay for android. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 72–81.

[7] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=kxnoqaisCT

[8] Jungjae Lee, Dongjae Lee, Chihun Choi, Youngmin Im, Jaeyoung Wi, Kihong Heo, Sangeun Oh, Sunjae Lee, and Insik Shin. 2025. VeriSafe Agent: Safeguarding Mobile GUI Agent via Logic-based Action Verification. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 817–831. doi:10.1145/3680207.3765248

[9] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. 2024. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1119–1133.

[10] Toby Jia-Jun Li, Amos Azaria, and Brad A Myers. 2017. SUGILITE: creating multimodal smartphone automation by demonstration. In *Proceedings of the 2017 CHI conference on human factors in computing systems*. 6038–6049.

[11] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19498–19508.

[12] Zibin Liu, Cheng Zhang, Xi Zhao, Yunfei Feng, Bingyu Bai, Dahu Feng, Erhu Feng, Yubin Xia, and Haibo Chen. 2025. Beyond Training: Enabling Self-Evolution of Agents with MOBIMEM. *arXiv preprint arXiv:2512.15784* (2025).

[13] Maestro. 2026. Maestro. End-to-End UI Testing for Mobile and Web. https://maestro.dev/.

[14] openatx. 2026. uiautomator2: Android Uiautomator2 Python Wrapper. https://github.com/openatx/uiautomator2.

[15] Antti Oulasvirta, Tye Rattenbury, Lingyi Ma, and Eeva Raita. 2012. Habits make smartphone use more pervasive. *Personal and Ubiquitous computing* 16, 1 (2012), 105–114.

[16] Mingyu Ouyang, Kevin Qinghong Lin, Mike Zheng Shou, and Hwee Tou Ng. 2026. FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection. *arXiv preprint arXiv:2601.03928* (2026).

[17] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=il5yUQsrjC

[18] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. 2023. Android in the wild: A large-scale dataset for android device control. *arXiv preprint arXiv:2307.10088* (2023).

[19] Shizuo Tian, Hao Wen, Yuxuan Chen, Jiacheng Liu, Shanhui Zhao, Guohong Liu, Ju Ren, Yunxin Liu, and Yuanchun Li. 2025. AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. *arXiv preprint arXiv:2512.10371* (2025).

[20] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling conversational interaction with mobile ui using large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–17.

[21] Yuan Wang, Mingyu Li, and Haibo Chen. 2026. From Imperative to Declarative: Towards LLM-friendly OS Interfaces for Boosted Computer-Use Agents. In *Proceedings of the 21st European Conference on Computer Systems*. 296–310.

[22] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th annual international conference on Mobile computing and networking*. 543–557.

[23] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, et al. 2025. Autodroid-v2: Boosting slm-based gui agents via code generation. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 223–235.

[24] Tingxin Yan, David Chu, Deepak Ganesan, Aman Kansal, and Jie Liu. 2012. Fast app launching for mobile devices using predictive user context. In *Proceedings of the 10th international conference on Mobile systems, applications, and services*. 113–126.

[25] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. 2024. Large Language Model-Brained GUI Agents: A Survey. *arXiv preprint arXiv:2411.18279* (2024).

[26] Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, et al. 2025. Ufo2: The desktop agentos. *arXiv preprint arXiv:2504.14603* (2025).

[27] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–20.

[28] Li Zhang, Longxi Gao, and Mengwei Xu. 2025. Does chain-of-thought reasoning help mobile gui agent? an empirical study. *arXiv preprint arXiv:2503.16788* (2025).

[29] Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesataporn-wongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. 2026. Actionengine: From reactive to programmatic gui agents via state machine memory. *arXiv preprint arXiv:2602.20502* (2026). https://arxiv.org/abs/2602.20502